

오늘의 작업기록 — 2026년 7월 20일

m106 완결 — 6379를 내 손으로 열다

(코드는 10분, 그 앞뒤 환경 마찰 4연쇄가 본체였던 날 — 그리고 "로컬 통과 ≠ 원격 통과"를 눈으로 본 날)

[문서 규칙] 이 파일은 발췌 없이 풀코드·풀출력으로 기재한다. 명령의 결과 화면, 파일 전체 내용, grep 판정 출력을 자르지 않고 전부 실는다 — 나중에 이 파일만 봐도 재현·이해가 되도록, 코드가 처음 등장하는 게 아니어도 병기한다(C축 원칙).

한눈에

항목	내용
목표	m106 Stage 1(포트 바인딩) 통과 — 25분 예정
결과	Stage 1 통과 ✓ Test passed. Congrats! · m106 done · 7월 관문 사실상 완주
실제 소요	환경 마찰 4연쇄로 계획 초과 — 근데 그 마찰들이 오늘의 진짜 수확
통과 커밋	eb0f749 (해제, 원격 실패) → 76c41a0 (preview 플러그 제거, 이게 통과판) — CodeCrafters git 서버(master 브랜치)
★ 핵심 코드	new ServerSocket(6379) + serverSocket.accept() — Spring이 대신 열던 포트를 처음 직접
★ 오늘의 교훈	로컬 컴파일 통과 ≠ 원격 빌드 통과 — preview 플러그가 로컬 JDK21선 안 걸리고 CodeCrafters(25 전제)가 잡아냄
마찰 4연쇄	① Maven 미설치 ② 자바 25↔21 불일치 ③ cd 스코프 재발(3번째) ④ --enable-preview 잔존
부수 확정	update.py = 매분 크론 자동봇 — 7/19 커밋 2개·"nothing to commit" 미스터리 전부 해명
잔디	이 로그 파일 + m106 done(크론이 자동 push)
다음	m107 월말 점검(관문1 조기 총족) · 밀린 복기 큐(7/5 원자성 재현 + 문답집 빈화면)

Stage 1의 코드는 "주석 풀기"라 손이 움직인 건 10분도 안 된다. 그런데 오늘 시간을 다 먹은 건 그 코드가 실제로 돌기까지 걸린 환경 장벽 넷이었고 — 정확히 그게 "만들어서 안다"의 실체다. 튜토리얼이 안 알려주는, 남의 템플릿과 내 환경이 안 맞을 때 어디를 어떻게 고치는가. ledger-api(Gradle)에선 안 겪던 Maven 특유의 지점들이 오늘 처음 나왔다.

0. 오늘의 위치 — 3일 미룬 조각의 완결, 그리고 다른 결로의 진입

7/15에 예약해둔 m106을, 수련회(7/17~19)를 거치며 계정→clone 명령→clone 실행으로 세 마디 쪼개 전진시켰고(부록 참조), 오늘 마지막 마디(정찰+코드+통과)를 닫았다.

날	m106 진행	성격
7/17	계정 + clone 명령 확보	착수 조각 (미지수 큰 구간 선제거)
7/18	clone 실행 → 레포 안착	착수 조각 (보는 주소 vs 받는 주소)
7/19	(복구만, 작업기록 없음)	원위치
7/20 (오늘)	정찰 + 코드 + Stage 1 통과	완결

이건 예외 번역 장(ledger-api, Spring)과 완전히 다른 결이다 — 지금까지진 Spring이 다 해주던 것 위에 로직을 엮었는데, 오늘은 그 아래층(Spring이 감춰주던 소켓)을 직접 열었다. 8월 Layer 0(저수준)의 위밍업이고, 이 포폴 정체성("만들어서 안다") 그 자체다.

1. 정찰 — 스케폴드 전문을 읽다 (어제 미룬 것)

cat src/main/java/Main.java 로 열었다. [업데이트: 아래는 발췌가 아니라 CodeCrafters가 준 Main.java 원본 전문이다 — 주석 처리된 스케폴드 상태 그대로.]

```
import java.io.IOException;
import java.net.ServerSocket;
import java.net.Socket;

public class Main {
    public static void main(String[] args){
        // You can use print statements as follows for debugging, they'll be visible when running tests.
        System.out.println("Logs from your program will appear here!");

        // Uncomment the code below to pass the first stage
        // ServerSocket serverSocket = null;
        // Socket clientSocket = null;
        // int port = 6379;
        // try {
        //     serverSocket = new ServerSocket(port);
        //     // Since the tester restarts your program quite often, setting SO_REUSEADDR
        //     // ensures that we don't run into 'Address already in use' errors
        //     serverSocket.setReuseAddress(true);
        //     // Wait for connection from client.
        //     clientSocket = serverSocket.accept();
        // } catch (IOException e) {
        //     System.out.println("IOException: " + e.getMessage());
        // } finally {
        //     try {
        //         if (clientSocket != null) {
        //             clientSocket.close();
        //         }
        //     } catch (IOException e) {
        //         System.out.println("IOException: " + e.getMessage());
        //     }
        // }
    }
}
```

핵심 두 이름을 찾았다: `new ServerSocket(port)` (포트를 여는 부품)과 `serverSocket.accept()` (클라이언트를 기다리는 줄). 나머지는 예외 처리(try/catch)와 자원 정리(finally의 close)라는 표준 골격.

정찰을 어제(7/18) 일부터 안 했던 이유가 오늘 맞아떨어졌다 — 이 코스는 "읽으며 주석 해제"(Study and uncomment)라 정찰(무슨 코드인지 읽기)과 실행(풀고 통과)이 같은 호흡이어야 한다. 어제 정찰만 땔으면 오늘 또 읽어야 했다. 구조(Is)는 지도라 어제 봐도 뻔했지만, 내용은 오늘 코드와 붙여서 봤다.

2. 마찰 ① — Maven이 없다

before 빌드를 확인하려는데:

```
root@ledger:~/codecrafters-redis-java# mvn -q compile
Command 'mvn' not found, but can be installed with:
apt install maven
```

ledger-api는 Gradle(`./gradlew`)이라 프로젝트에 빌드 래퍼가 달려왔는데, 이 레포는 **Maven**인데 `mvn` 명령 자체가 시스템에 없다. SDKMAN으로 한 줄:

```
root@ledger:~/codecrafters-redis-java# sdk install maven

Downloading: maven 3.9.16

In progress...

##### 100.0%

Installing: maven 3.9.16
Done installing!

Setting maven 3.9.16 as default.
```

7/6에 gradlew가 처음 의존성 받던 것과 같은 종류 — 도구 최초 설치는 한 번만 겪는 통과이레다. 첫 **Maven 프로젝트**라는 표식이기도 하다(7/18에 pom.xml 보고 예고했던 그 지점).

3. 마찰 ② — 자바 25 vs JDK 21

Maven 깔고 다시 빌드하니 이번엔 컴파일 에러. [업데이트: 아래는 `tail -5` 로 자른 결론이 아니라, 원인이 드러나는 줄까지 포함한 실제 에러 출력이다.]

```
root@ledger:~/codecrafters-redis-java# mvn compile 2>&1 | grep -A3 "ERROR\|BUILD" | head -20
[INFO] BUILD FAILURE
[INFO] -----
[INFO] Total time: 2.986 s
[INFO] Finished at: 2026-07-20T11:11:37Z
--
[ERROR] Failed to execute goal org.apache.maven.plugins:maven-compiler-plugin:3.15.0:compile (default-compile) on project codecrafters-redis:
Fatal error compiling: error: invalid target release: 25 -> [Help 1]
[ERROR]
[ERROR] To see the full stack trace of the errors, re-run Maven with the -e switch.
[ERROR] Re-run Maven using the -X switch to enable full debug logging.
```

원인 추적 — "가정 말고 확인" (설치된 JDK와 pom.xml 요구를 나란히 봄):

```
root@ledger:~/codecrafters-redis-java# java -version 2>&1 && echo "=====" && grep -i "release\|source\|target\|java.version" pom.xml
openjdk version "21.0.11" 2026-04-21 LTS
OpenJDK Runtime Environment Temurin-21.0.11+10 (build 21.0.11+10-LTS)
OpenJDK 64-Bit Server VM Temurin-21.0.11+10 (build 21.0.11+10-LTS, mixed mode, sharing)
=====
<maven.compiler.source>25</maven.compiler.source>
<maven.compiler.target>25</maven.compiler.target>
<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
<java.version>25</java.version>
```

명확해졌다 — VPS엔 JDK 21이 깔렸는데, CodeCrafters 템플릿의 pom.xml은 자바 25로 컴파일하라고 박아뒀다. 코드 문제가 아니라 버전 불일치다. 남의 템플릿이 최신 자바를 기본값으로 가정한 것.

방향 판단: 25를 새로 까느냐 vs pom을 21로 낮추느냐. 후자를 택했다 — Stage 1 코드(소켓 열기)는 자바 25 기능을 하나도 안 쓰고, 21은 최신 LTS라 안정적이며, 25 설치느 오래 걸리고 과하다.

```
root@ledger:~/codecrafters-redis-java# sed -i 's/>25</>21</g' pom.xml && grep -i "release\|source\|target\|java.version" pom.xml
<maven.compiler.source>21</maven.compiler.source>
<maven.compiler.target>21</maven.compiler.target>
<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
<java.version>21</java.version>
```

>25< 패턴을 전부 >21< 로 바꿨고, grep으로 세 항목(source/target/java.version) 전부 21 확인. 재빌드:

```
root@ledger:~/codecrafters-redis-java# mvn -q compile
root@ledger:~/codecrafters-redis-java#
```

-q (조용 모드)라 성공하면 아무 출력이 없다 — 로컬 컴파일 통과. 이때는 이게 끝인 줄 알았다 — 아니었다(마찰 ④).

4. 마찰 ③ — cd 스코프, 세 번째 재발

스캐폴드를 nano로 해제하고 (§ 5) 저장한 뒤, 컴파일하려는데:

```
root@ledger:~# mvn -q compile 2>&1 | tail -5
[ERROR] To see the full stack trace of the errors, re-run Maven with the -e switch.
[ERROR] Re-run Maven using the -X switch to enable full debug logging.
[ERROR]
[ERROR] For more information about the errors and possible solutions, please read the following articles:
[ERROR] [Help 1] http://cwiki.apache.org/confluence/display/MAVEN/MissingProjectException
```

또 위치 문제였다. 프롬프트를 보니 root@ledger:~# = 홈이지 ~/codecrafters-redis-java 가 아니었다. 세션이 새로 열리면서 위치가 홈으로 돌아갔고, 거기엔 pom.xml이 없어 Maven이 MissingProjectException (프로젝트 없음)을 뱉은 거다.

이건 7/12·7/14에 이어 세 번째 같은 계열의 실수다 — cd의 효력 범위를 착각하는 것(7/14엔 cd A && cmd & 의 백그라운드 스코프, 오늘은 세션 전환으로 인한 위치 리셋). 원인은 조금씩 다르지만 증상은 같다: "내가 지금 어느 디렉토리에 있는가"를 확인 안 하고 명령을 침.

```
root@ledger:~# cd ~/codecrafters-redis-java && mvn -q compile
root@ledger:~/codecrafters-redis-java#
```

cd를 붙여서 다시 치니 조용히 통과. 교훈 고정: 빌드·실행 명령 앞엔 위치를 명령에 붙여서(cd X && ...) 치거나, 최소한 프롬프트의 경로를 눈으로 확인한다. 세 번 나왔으면

우연이 아니라 절차의 구멍이다 — 못 박는다.

5. 코드 — 스케폴드 해제 (오늘의 본체, 그러나 가장 쉬웠던 부분)

nano로 열어 주석 블록 각 줄의 `//` 를 벗겼다. [업데이트: 아래는 해제 후 Main.java 전문이다 — 발췌 아님, 커밋 `76c41a0` 에 실린 최종 상태.]

```
import java.io.IOException;
import java.net.ServerSocket;
import java.net.Socket;

public class Main {
    public static void main(String[] args){
        // You can use print statements as follows for debugging, they'll be visible when running tests.
        System.out.println("Logs from your program will appear here!");

        ServerSocket serverSocket = null;
        Socket clientSocket = null;
        int port = 6379;
        try {
            serverSocket = new ServerSocket(port);
            // Since the tester restarts your program quite often, setting SO_REUSEADDR
            // ensures that we don't run into 'Address already in use' errors
            serverSocket.setReuseAddress(true);
            // Wait for connection from client.
            clientSocket = serverSocket.accept();
        } catch (IOException e) {
            System.out.println("IOException: " + e.getMessage());
        } finally {
            try {
                if (clientSocket != null) {
                    clientSocket.close();
                }
            } catch (IOException e) {
                System.out.println("IOException: " + e.getMessage());
            }
        }
    }
}
```

(설명용 `//` 주석 두 줄 — `// Since the tester...` 와 `// Wait for connection...` — 은 코드가 아니라 안내라 남겨둬도 무관하다. 벗겨낸 건 `//` 접두사가 붙어 **코드를 막고 있던** 줄들이다.)

두 줄에 멈춰 새긴 것

`serverSocket = new ServerSocket(port)` — **포트를 연다**. 7/6에 "마법 목록 ①"으로 적었던 그것 — *Tomcat이 8080을 열고 대기* — 의 아래층이다. mini-ledger·ledger-api 내내 Spring이 대신 열어주던 그 행위를, 오늘 처음 **내 코드 한 줄**이 한다. 6379는 Redis의 표준 포트다.

`clientSocket = serverSocket.accept()` — **기다린다. 이게 서버의 심장이다**. 7/6 Q1에서 "서버의 존재 이유는 기다림"이라고 글로 답했었다. 오늘 그 기다림이 **코드 한 줄**로 처음 보였다. 이 줄에 도달하면 프로그램이 **멈춘다** — 클라이언트가 접속해 올 때까지. 그게 **blocking**이고, 서버가 "떠 있다"는 말의 실제 의미다. CPU를 태우며 도는 게 아니라, 이 줄에서 조용히 멈춰 기다리는 것.

`setReuseAddress(true)` 는 테스터가 프로그램을 자주 재시작해도 "Address already in use"가 안 나게 하는 것(주석이 그렇게 설명). 7/7·7/12에 서버 내리고 포트 빠지는 시차를 겪었던 그 문제의, 코드 레벨 예방책인 셈이다.

6. 마찰 ④ — ★ 로컬은 통과했는데 원격이 실패했다 (오늘의 진짜 교훈)

해제한 코드가 로컬에서 컴파일됐으니 (§ 4 끝) push했다:

```
root@ledger:~/codecrafters-redis-java# git add . && git commit -m "bind to port 6379 (stage 1)" && git push origin master
[master eb0f749] bind to port 6379 (stage 1)
 2 files changed, 23 insertions(+), 27 deletions(-)
```

CodeCrafters가 원격 빌드를 돌렸고 — **실패했다**. [업데이트: 원격 빌드 로그 중 실패 원인이 드러나는 핵심 구간 전문.]

```
remote: [build] > [ERROR] COMPILATION ERROR :
remote: [build] > [ERROR] invalid source release 21 with --enable-preview
remote: [build] > (preview language features are only supported for release 25)
remote: [build] > [ERROR] Failed to execute goal org.apache.maven.plugins:maven-compiler-plugin:3.13.0:compile (default-compile) on project
codecrafters-redis: Compilation failure
remote: [build] > [ERROR] invalid source release 21 with --enable-preview
remote: [build] > [ERROR] (preview language features are only supported for release 25)
remote: [build] > [ERROR] -> [Help 1]
remote: [build] Error: process "/bin/sh -c .codecrafters/compile.sh" did not complete successfully: exit code: 1.
remote: [build] Build failed. Check the logs above for the reason.
```

여기가 오늘 가장 값진 지점이다. 로컬 `mvn -q compile` 은 조용히 통과했는데(§ 3 끝), 원격은 같은 코드를 거부했다. 왜?

원인: pom.xml에 내가 못 본 게 하나 더 있었다. grep으로 위치를 찍었다:

```
root@ledger:~/codecrafters-redis-java# grep -n "enable-preview\|compilerArgs\|release" pom.xml
24:         <compilerArgs>
25:             <arg>--enable-preview</arg>
26:         </compilerArgs>
```

`--enable-preview` 는 **release 25에서만** 되는 "미리보기 기능 켜기" 플래그다. § 3에서 25→21로 낮췄지만 이 플래그는 그대로 남아서, "21인데 preview 켜라"는 모순이 됐다.

그런데 왜 로컬은 통과했나? — 이게 핵심 통찰이다. 로컬 JDK 21은 preview 기능을 실제로 안 쓰니 그 플래그를 **무해하게 흘려보냈다**. 반면 CodeCrafters 빌드 환경은 (원래 25용이라) 그 플래그를 **엄격히 검사**해서 모순을 잡아냈다. 즉:

로컬 컴파일 성공이 원격 빌드 성공을 보장하지 않는다. 같은 코드, 같은 pom.xml인데 환경(JDK 버전, 빌드 엄격도)이 다르면 결과가 갈린다.

이건 실무의 핵심 함정 하나다 — "내 컴퓨터에선 되는데요(works on my machine)". 오늘 그걸 **책이 아니라 실패 로그로** 봤다. ledger-api는 로컬 = 실행 환경이 같아서(둘 다 이 VPS) 이런 어긋남이 안 났는데, CodeCrafters는 **로컬(내 VPS)과 채점(그들 CI)이 분리**돼 있어서 처음 드러난 것.

해결 — `compilerArgs` 블록 3줄 통째 제거:

```
root@ledger:~/codecrafters-redis-java# sed -i '/<compilerArgs>/,/<\|/compilerArgs>/d' pom.xml && grep -n "enable-
preview\|compilerArgs\|release\|source\|target" pom.xml
12:         <maven.compiler.source>21</maven.compiler.source>
13:         <maven.compiler.target>21</maven.compiler.target>
14:         <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
```

sed 범위 삭제(`/<compilerArgs>/,/<\|/compilerArgs>/d` = 여는 태그부터 닫는 태그까지)로 24~26줄이 사라졌다. grep 결과에 `enable-preview·compilerArgs`가 **아** **예** **안** **나오**고 source/target 21만 남았다 = 정확히 그 블록만 제거됨. Stage 1 코드는 preview 기능을 안 쓰니 이 블록 자체가 불필요했다 — 삭제가 정답. 로컬 재확인:

```
root@ledger:~/codecrafters-redis-java# mvn -q compile
root@ledger:~/codecrafters-redis-java#
```

7. 통과 — Stage 1 초록

```
root@ledger:~/codecrafters-redis-java# git add . && git commit -m "remove --enable-preview flag (not needed for JDK 21)" && git push origin master
[master 76c41a0] remove --enable-preview flag (not needed for JDK 21)
 1 file changed, 3 deletions(-)
```

이번엔 원격 빌드 로그가 끝까지 갔다. [업데이트: 빌드 성공부터 테스트 통과까지 핵심 구간 전문 — 반복되는 Step 로그는 생략하되 판정 줄은 원문 그대로.]

```
remote: [build] Build successful.
remote:
remote: [compile] Moved ../codecrafters/run.sh → ../your_program.sh
remote: [compile] Compilation successful.
remote:
remote: [tester::#JM1] Running tests for Stage #JM1 (Bind to a port)
remote: [tester::#JM1] $ ./your_program.sh
remote: [tester::#JM1] Connecting to port 6379...
remote: [your_program] Logs from your program will appear here!
remote: [tester::#JM1] Test passed.
remote:
remote: Test passed. Congrats!
remote:
remote: Mark step as complete in your browser.
To https://git.codecrafters.io/1e6aca72db41e76f
eb0f749..76c41a0  master -> master
```

Connecting to port 6379... → Test passed. — 이 두 줄이 전부다. 테스터가 **6379로 실제 접속을 시도했고, 내 프로그램의 `accept()` 가 그걸 받아냈다.** § 5에서 "이 줄에서 멈춰 기다린다"고 새긴 그 **`accept()`** 가, 방금 진짜 클라이언트(테스터)를 받은 것이다. 7/6에 글로 적은 "서버의 존재 이유는 기다림"이 코드로 돌아가 실증된 순간. 통과 커밋 **76c41a0**. (직전 실패 커밋 **eb0f749** 도 이력에 남는다 — "해제 → preview 제거"의 두 걸음이 정직하게 기록됨.)

8. 마일스톤 + 부수 확정 (update.py의 정체)

m106 done 처리 — roadmap.json 직접 수정. [업데이트: sed 명령 + 판정 출력 전문.]

```
root@ledger:~/backend-journey# sed -i '/"id": "m106"/,/"status"/ s/"status": "todo"/"status": "done"/' roadmap.json && grep -A3 '"id": "m106"'
roadmap.json
    "id": "m106",
    "title": "CodeCrafters 'Build Your Own Redis' 1단계",
    "weight": 2,
    "status": "done"
```

sed 범위 치환(**`/"id": "m106"/,/"status"/`** = m106 id 줄부터 그 다음 status 줄까지의 범위 안에서만 todo→done)이라 다른 마일스톤은 안 건드린다. JSON 무결성 확인:

```
root@ledger:~/backend-journey# python3 -c "import json; json.load(open('roadmap.json')); print('JSON OK!)"
JSON OK
```

★ 왜 **update.py**를 안 쓰고 **sed**로 직접 고쳤나 — 이번엔 정체가 확정됐다. 7/19에 "update.py가 인자를 안 받고 자동 커밋한다"까지는 알았는데, 오늘 VPS 진단 중 크론텍에서 진짜 원인을 봤다. [업데이트: 크론텍 원문.]

```
* * * * * cd /root/backend-journey && /usr/bin/python3 update.py >> /root/backend-journey/.cron.log 2>&1
```

update.py는 매 1분마다 크론으로 자동 실행되는 봇이다. 이 하나가 7/19의 두 미스터리를 다 설명한다: - "**7/19 커밋이 왜 두 개?**" → 매분 크론이 돌면서 로그 반영분과 로드맵 반영분이 다른 분(minute)에 걸려 커밋이 걸렸다. - "**m106 정정 후 수동 commit이 왜 nothing to commit?**" → sed로 고친 직후 1분 내에 크론이 먼저 커밋해버렸다(953cc07).

그러니 오늘도 **sed로 done만 찍으면 1분 내 크론이 자동 커밋·push**한다 — 수동 커밋 불필요. 이게 이 대시보드의 작동 원리다.

브라우저에서 CodeCrafters "Mark step as complete"도 눌러 그쪽 화면도 초록으로 맞췄다.

9. 로드맵 위치

- **m106 완결 = 7월 관문 사실상 완주.** 남은 7월 항목은 m107(월말 점검)뿐 — 관문1을 2주 앞당겨 충족한 상태 유지.
- **ledger-api 현재:** GET /health · POST /transfer(명등성·복식부기·3핸들러 JSON 에러). 예외 번역 장 완결(7/15) 이후 손 안 댄 — 오늘은 별개 트랙(Redis).
- **codecrafters-redis-java 현재:** Stage 1(포트 바인딩) 통과. Stage 2(PING/PONG, RESP 프로토콜 첫 발)는 자유 슬롯.
- **밀린 복기 큐(11일+):** 7/5분 원자성 트래이스(transfer 순서 종이에 + checkBalance 빈화면 재현) + 문답집 ★15문항 빈화면 즉답. m106 닫혔으니 다음 세션 첫 순서 후보.
- **8월 WAL 이유 3개 유지:** 원자성 한계(7/5) · 영속성 부재(7/12) · 롤백 불가(7/13).

10. 검증 질문 (다음 복기용 — 답 적지 말 것)

1. **`new ServerSocket(6379)`** 와 **`accept()`** 는 각각 무엇을 하나? 이 중 "서버의 심장"은 어느 쪽이고, 그 줄에서 프로그램은 어떤 상태가 되나(blocking의 의미)?
2. 이 두 줄이 ledger-api(Spring) 때와 근본적으로 다른 점은? Spring은 이걸 어디서 대신 해주고 있었나?
3. **`setReuseAddress(true)`** 는 왜 필요한가? 7/7·7/12에 겪은 어떤 현상과 연결되나?

- 4. 자바 25↔21 불일치는 어디서 났고(pom.xml의 어느 항목들), 25를 새로 까는 대신 21로 낮춘 판단의 근거 셋은?
- 5. ★ 로컬 `mvn compile` 은 통과했는데 CodeCrafters 원격 빌드는 왜 실패했나? `--enable-preview` 가 로컬 JDK21에선 통과하고 원격에선 걸린 이유는? 여기서 얻은 한 문장짜리 교훈은?
- 6. "로컬 = 실행 환경"이던 ledger-api와 "로컬(내 VPS) ≠ 채점(CI)"인 CodeCrafters의 구조 차이가, 왜 이 함정을 CodeCrafters에서만 드러나게 했나?
- 7. MissingProjectException은 왜 났나? cd 스코프 실수가 이번까지 세 번(7/12·7/14·7/20) 나온 공통 원인 한 문장과, 고정 절차는?
- 8. update.py를 안 쓰고 roadmap.json을 sed로 직접 고친 이유는? 크론탭의 어떤 줄이 7/19의 "커밋 2개"와 "nothing to commit"을 설명하나?
- 9. 통과 이력에 실패 커밋(eb0f749)과 성공 커밋(76c41a0)이 둘 다 남은 게 왜 정직한 기록인가?
- 0. Stage 1 통과 로그의 `Connecting to port 6379... → Test passed` 두 줄에서, 내 코드의 어느 줄이 그 "Connecting"을 받아낸 건가?

11. 회고

코드는 가장 쉬웠고, 그 코드를 돌아가게 만드는 게 오늘의 전부였다. 스캐폴드 주석 풀기는 10분도 안 걸렸다. 그런데 그 앞뒤로 Maven 설치, 자바 버전 불일치, 디렉토리 착각, preview 플러그까지 넷이 줄줄이 걸렸고 — 그걸 하나씩 진단하고 뚫는 게 오늘 시간의 대부분이었다. 이게 "만들어서 안다"의 민낯이다. 튜토리얼은 "주석을 푸세요"까지만 말하고, 그 사이에 낀 네 개의 벽은 안 알려준다. 그 벽을 직접 넘은 게 오늘 남는다.

가장 값진 건 마찰 ④였다. 로컬에서 `mvn compile` 이 조용히 통과했을 때 나는 끝난 줄 알았는데, CodeCrafters가 같은 코드를 거부했다. 원인을 파보니 preview 플러그 하나 — 로컬 JDK21은 그걸 안 쓰니 흘러보냈고, 원격은 엄격히 검사해 잡았다. "내 컴퓨터에선 되는데요"라는 그 유명한 함정을, mim으로가 아니라 실패 로그로 봤다. ledger-api에선 로컬과 실행이 같은 VPS라 이런 어긋남이 원천적으로 없었는데, CodeCrafters는 내 VPS(로컬)와 그들 CI(채점)가 분리돼 있어서 처음 드러났다. 로컬 통과는 필요조건이지 충분조건이 아니다 — 이 한 문장이 오늘의 핵이다.

정확하게 남길 것 하나 더. cd 스코프 실수가 세 번째 나왔다(7/12, 7/14, 오늘). 원인은 조금씩 달라도 증상은 하나 — 내가 지금 어느 폴더에 있는지 확인 안 하고 명령을 친다는 것. 세 번이면 우연이 아니라 절차의 구멍이다. 빌드·실행 명령 앞엔 cd를 붙여서 치는 걸로 못 막는다. 그리고 update.py의 정제도 오늘에야 완전히 잡혔다 — 매번 크론으로 도는 자동봇. 7/19에 나를 어리둥절하게 했던 "커밋 두 개"와 "nothing to commit"이 이 한 줄로 다 설명됐다. 모르고 쓰던 자동화의 뚜껑을 열어본 셈이다.

m106이 닫혔다. 7월 관문은 2주 앞당겨 사실상 완주했고, 남은 건 월말 점검뿐이다. 오늘 처음으로 Spring 없이 포트를 열었다 — 6379에서 `accept()` 가 클라이언트를 기다리다 받아냈다. 다음은 Stage 2, RESP 프로토콜의 첫 발(PING에 PONG). 서버가 "듣기"에서 "대답하기"로 넘어가는 자리다. 그 전에 밀린 복기부터 청산하는 게 순서일 수도 있고 — 그건 다음 세션 컨디션 보고 정한다.

한 줄: Spring 없이 6379를 처음 직접 열어 Stage 1 통과(accept()가 테스트를 받아냄) — 코드 10분, 그 코드를 돌아가게 한 환경 마찰 4연쇄가 본체였고, 그중 "로컬 통과 ≠ 원격 통과"(preview 플러그가 CI에서만 걸림)를 실패 로그로 배운 게 오늘의 핵. m106 완결 = 7월 사실상 완주.

부록 — 수련회 3일 백필 요약 (7/17~19, 로그 파일 복원)

아래 3일은 당시 작업기록을 별도 파일로 남겼으나(7/17·18) logs 반영이 늦어, 오늘 커밋에 함께 실는 복원 요약이다. 7/19는 애초에 파일을 안 만든 날.

7/17 (수련회 첫날) — m106 착수: 계정 + clone 명령 확보. 세 번째 통째 이월 대신 맨 앞 조각만 뗐다. codecrafters.io → GitHub 로그인 → Build Your Own Redis → Java → **clone 명령 확보**까지. 완결점을 "계정 생성"이 아니라 "clone 명령 확보"로 잡은 이유: 코스·언어를 선택해야 clone 주소가 발급되므로, 계정만 만들면 반쪽이다. 쪼갬 기준 = "오늘 몇 분"이 아니라 **"내일이 어디서 시작하나"**(미지수 큰 모바일 UI 구간을 선제거). 주의 박음: 기본 브랜치 **master**, CodeCrafters push는 GitHub 잔디 안 찍힘.

7/18 (수련회 둘째날) — clone 실행 → 레포 VPS 안착. 어제 명령을 실제 실행 → `codecrafters-redis-java` 안착. 마찰: **"보는 주소(app.codecrafters.io) vs 받는 주소(git.codecrafters.io)"** 구분 — 브라우저 URL이 아니라 화면 본문 "Clone the repository" 복붙칸이 답. 구조만 보고(내용 안 읽) 미리 파악: **pom.xml → 빌드는 Maven**(첫 Maven), Main.java 위치. 정찰을 일부러 복귀 후로 미룸(정찰+코드는 같은 호흡). — 오늘 pom.xml의 자바 25·preview 플러그로 고생한 게, 그때 "pom.xml = Maven"이라고만 보고 내용은 안 열었던 그 파일이었다.

7/19 (수련회 셋째날) — 복구만, 작업기록 없음. 실수 복구 둘: ① `logs/2026-07-17.md` 를 터미널에 붙여넣다 비운 것 → `git checkout` 복구(커밋 안 땀어서 GitHub 원본 무사) ② roadmap.json에서 m106이 실수로 done 찍힌 것 → todo로 정정. 둘 다 새 진도가 아니라 원위치라 남길 실질이 없어 파일을 안 만들었다(7/11 원칙 — 부풀리지 않는다). — 오늘 그 m106을 진짜로 통과시켜 정당하게 done을 찍었으니, 7/19의 정정이 오늘로 완결된 셈이다.